Milestone 7 Bewijs in PDF

Romeo Weyns & Jonas Friedhoff - INF202

S1 – Romeo Weyns

Techniek: RANGE-INTERVAL partitionering op TICKET_PURCHASE_DATE
D-tabel: TICKET

Stap 1: Voorbereiding

1.1 Tabelinfo en grootte van de brontabel (VOOR partitionering)

De brontabel voor S1 is TICKET. Voor de partitionering werd het rij-aantal en de fysieke grootte gecontroleerd om te voldoen aan de minimumvereisten van de opdracht (≥ 400 000 rijen, ≥ 60 MB).

Controle rij-aantal:

```
SELECT COUNT(*) AS ticket_count
FROM Ticket;
```

	TICKET_COUNT
1	400000

Grootte van de tabel:

```
SELECT segment_name, segment_type,
       SUM(bytes / 1024 / 1024) AS MB,
       (SELECT COUNT(*) FROM Ticket) AS table_count
FROM dba_segments
WHERE segment_name = 'TICKET'
GROUP BY segment_name, segment_type;
```

	SEGMENT_NAME	SEGMENT_TYPE	MB	TABLE_COUNT
1	TICKET	TABLE	80	400000

De TICKET-tabel voldoet aan de minimumvereisten van 400 000 rijen en 60 MB aan realistische data.

1.2 Statistieken berekenen

Alvorens het explain plan te genereren, werden de statistieken berekend zodat de Oracle optimizer over accurate informatie beschikt:

```
BEGIN
    DBMS_STATS.GATHER_TABLE_STATS(USER, 'TICKET');
END;
/
```

1.3 Domeinspecifieke aggregatiequery

De bedrijfsvraag luidt: Wat is de omzetanalyse per festival en tickettype voor tickets aangekocht in het kalenderjaar 2025?

De query groepeert op Festival_Name en Ticket_Type en beperkt het bereik tot aankoopdatums tussen 01-01-2025 en 31-12-2025. Dit is slechts één van de zes aanwezige jaargangen (2021–2026), zodat de WHERE-clausule een beperkt fragment van de volledige tabel bevraagt, ideaal om het effect van partitionering aan te tonen.

```
SELECT f.Festival_Name                AS Festival,
       t.Ticket_Type                  AS Tickettype,
       COUNT(t.Ticket_ID)             AS Aantal_Tickets,
       ROUND(AVG(t.Ticket_Price), 2)  AS Gemiddelde_Prijs_EUR,
       SUM(t.Ticket_Price)            AS Totale_Omzet_EUR,
       MIN(t.Ticket_Price)            AS Laagste_Prijs,
       MAX(t.Ticket_Price)            AS Hoogste_Prijs
FROM   Ticket t
       JOIN Festival f ON f.Festival_ID = t.Festival_ID
WHERE  t.Ticket_Purchase_Date
           BETWEEN DATE '2025-01-01' AND DATE '2025-12-31'
GROUP  BY f.Festival_Name, t.Ticket_Type
ORDER  BY Totale_Omzet_EUR DESC;
```

	FESTIVAL	TICKETTYPE	AANTAL_TICKETS	GEMIDDELDE_PRIJS_EUR	TOTALE_OMZET_EUR	LAAGSTE_PRIJS	HOOGSTE_PRIJS
1	Boom Live_0105_0000000…	Day	3987	278.44	1110145	50	500
2	Boom Live_0105_0000000…	Full Festival	4001	273.98	1096181	50	500
3	Boom Live_0105_0000000…	Weekend	3933	274.74	1080548	50	500
4	Boom Live_0105_0000000…	Combi	3874	274.87	1064849	50	500
5	Boom Live_0105_0000000…	VIP	3773	274.67	1036339	50	500
6	Berlin Open Air_0058_0…	Weekend	2939	275.64	810113	50	500
7	Berlin Open Air_0058_0…	Combi	2866	275.14	788555	50	500
8	Berlin Open Air_0058_0…	Day	2866	274.52	786782	50	500
9	Berlin Open Air_0058_0…	Full Festival	2750	277.42	762908	50	500
10	Berlin Open Air_0058_0…	VIP	2798	272.02	761125	50	500
11	Werchter Live_0068_000…	Full Festival	2565	272.75	699614	50	500
12	Werchter Live_0068_000…	Combi	2510	278.28	698490	50	500
13	Werchter Live_0068_000…	VIP	2488	274.44	682811	50	500
14	Werchter Live_0068_000…	Day	2448	276.65	677232	50	500
15	Werchter Live_0068_000…	Weekend	2385	274.1	653722	50	500
16	Barcelona Festival_020…	Combi	2200	277.14	609710	50	500
17	Barcelona Festival_020…	VIP	2197	274.93	604025	50	500
18	Barcelona Festival_020…	Day	2144	274.33	588158	50	500
19	Barcelona Festival_020…	Weekend	2132	275.32	586986	50	500
20	Barcelona Festival_020…	Full Festival	2119	271.63	575586	50	500
21	Hasselt Open Air_0272_…	VIP	2000	272.32	544632	50	500
22	Hasselt Open Air_0272_…	Full Festival	1982	273.45	541983	50	499
23	Hasselt Open Air_0272_…	Day	1961	274.75	538785	50	500
24	Hasselt Open Air_0272_…	Weekend	1953	273.12	533397	50	500
25	Hasselt Open Air_0272_…	Combi	1919	277.32	532170	50	500
26	Paris Open Air_0016_00…	VIP	1707	277.66	473958	50	500
27	Paris Open Air_0016_00…	Full Festival	1656	274.14	453984	50	500
28	Paris Open Air_0016_00…	Day	1667	272.06	453530	50	500
29	Paris Open Air_0016_00…	Combi	1625	272.92	443487	50	500
30	Glastonbury Open Air_0…	Full Festival	1592	275.8	439074	50	500
31	Paris Open Air_0016_00…	Weekend	1614	272	439016	50	500
32	Glastonbury Open Air_0…	Day	1597	272.62	435381	50	500

200 rows

Stap 2: Analyse Voor optimalisatie

2.1 Explain plan Voor partitionering (Plan hash: 2956872105)

De query werd uitgevoerd op de niet-gepartitioneerde TICKET-tabel. Het explain plan werd gegenereerd via:

```
EXPLAIN PLAN SET STATEMENT_ID = 'M7_S1_VOOR' FOR
SELECT f.Festival_Name                AS Festival,
       t.Ticket_Type                  AS Tickettype,
       COUNT(t.Ticket_ID)             AS Aantal_Tickets,
       ROUND(AVG(t.Ticket_Price), 2)  AS Gemiddelde_Prijs_EUR,
       SUM(t.Ticket_Price)            AS Totale_Omzet_EUR,
       MIN(t.Ticket_Price)            AS Laagste_Prijs,
       MAX(t.Ticket_Price)            AS Hoogste_Prijs
FROM Ticket t
       JOIN Festival f ON f.Festival_ID = t.Festival_ID
WHERE t.Ticket_Purchase_Date
       BETWEEN DATE '2025-01-01' AND DATE '2025-12-31'
GROUP BY f.Festival_Name, t.Ticket_Type
ORDER BY Totale_Omzet_EUR DESC;
```

Execution plan

PLAN_TABLE_OUTPUT

```
Plan hash value: 2956872105

-----------------------------------------------------------------------------------------------------------------------
| Id  | Operation                          | Name     | Rows  | Bytes | Cost (%CPU)| Time     |    TQ  |IN-OUT| PQ Distrib |
-----------------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                   |          |   191 | 24066 |  1530   (1)| 00:00:01 |        |      |            |
|   1 |  PX COORDINATOR                    |          |       |       |            |          |        |      |            |
|   2 |   PX SEND QC (ORDER)               | :TQ10004 |   191 | 24066 |  1530   (1)| 00:00:01 |  Q1,04 | P->S | QC (ORDER) |
|   3 |    SORT ORDER BY                   |          |   191 | 24066 |  1530   (1)| 00:00:01 |  Q1,04 | PCWP |            |
|   4 |     PX RECEIVE                     |          |   191 | 24066 |  1530   (1)| 00:00:01 |  Q1,04 | PCWP |            |
|   5 |      PX SEND RANGE                 | :TQ10003 |   191 | 24066 |  1530   (1)| 00:00:01 |  Q1,03 | P->P | RANGE      |
|   6 |       HASH GROUP BY                |          |   191 | 24066 |  1530   (1)| 00:00:01 |  Q1,03 | PCWP |            |
|   7 |        PX RECEIVE                  |          |   191 | 24066 |  1530   (1)| 00:00:01 |  Q1,03 | PCWP |            |
|   8 |         PX SEND HASH               | :TQ10002 |   191 | 24066 |  1530   (1)| 00:00:01 |  Q1,02 | P->P | HASH       |
|   9 |          HASH GROUP BY             |          |   191 | 24066 |  1530   (1)| 00:00:01 |  Q1,02 | PCWP |            |
|* 10 |           HASH JOIN                |          |   191 | 24066 |  1528   (1)| 00:00:01 |  Q1,02 | PCWP |            |
|  11 |            VIEW                    | VW_GBF_6 |   300 | 14700 |     2   (0)| 00:00:01 |  Q1,02 | PCWP |            |
|  12 |             PX BLOCK ITERATOR      |          |   300 | 11100 |     2   (0)| 00:00:01 |  Q1,02 | PCWC |            |
|  13 |              TABLE ACCESS FULL     | FESTIVAL |   300 | 11100 |     2   (0)| 00:00:01 |  Q1,02 | PCWP |            |
|  14 |            PX RECEIVE              |          |   191 | 14707 |  1526   (1)| 00:00:01 |  Q1,02 | PCWP |            |
|  15 |             PX SEND BROADCAST      | :TQ10001 |   191 | 14707 |  1526   (1)| 00:00:01 |  Q1,01 | P->P | BROADCAST  |
|  16 |              VIEW                  | VW_GBC_5 |   191 | 14707 |  1526   (1)| 00:00:01 |  Q1,01 | PCWP |            |
|  17 |               HASH GROUP BY        |          |   191 |  4584 |  1526   (1)| 00:00:01 |  Q1,01 | PCWP |            |
|  18 |                PX RECEIVE          |          |   191 |  4584 |  1526   (1)| 00:00:01 |  Q1,01 | PCWP |            |
|  19 |                 PX SEND HASH       | :TQ10000 |   191 |  4584 |  1526   (1)| 00:00:01 |  Q1,00 | P->P | HASH       |
|  20 |                  HASH GROUP BY     |          |   191 |  4584 |  1526   (1)| 00:00:01 |  Q1,00 | PCWP |            |
|  21 |                   PX BLOCK ITERATOR|          |  181K |  4255K|  1523   (1)| 00:00:01 |  Q1,00 | PCWC |            |
|* 22 |                    TABLE ACCESS FULL| TICKET   |  181K |  4255K|  1523   (1)| 00:00:01 |  Q1,00 | PCWP |            |
-----------------------------------------------------------------------------------------------------------------------
```

```
Query Block Name / Object Alias (identified by operation id):
-------------------------------------------------------------

   1 - SEL$0B6E4868
  11 - SEL$4ED5C62B / VW_GBF_6@SEL$381ECC48
  13 - SEL$4ED5C62B / F@SEL$1
  16 - SEL$8B9A55F1 / VW_GBC_5@SEL$6C2C6143
  17 - SEL$8B9A55F1
  22 - SEL$8B9A55F1 / T@SEL$1
```

Predicate Information

```
Predicate Information (identified by operation id):
---------------------------------------------------

  10 - access("ITEM_1"="ITEM_1")
  22 - filter("T"."TICKET_PURCHASE_DATE">=TO_DATE(' 2025-01-01 00:00:00', 'syyyy-mm-dd hh24:mi:ss') AND
              "T"."TICKET_PURCHASE_DATE"<=TO_DATE(' 2025-12-31 00:00:00', 'syyyy-mm-dd hh24:mi:ss'))
```

Column Projection Information

```
Column Projection Information (identified by operation id):
-----------------------------------------------------------

   1 - SUM()[22], "ITEM_6"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], NVL(SUM(),0)[22], ROUND(SUM()/NVL(SUM(),0),2)[22],
       MIN()[22], MAX()[22]
   2 - (#keys=0) SUM()[22], "ITEM_6"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], NVL(SUM(),0)[22],
       ROUND(SUM()/NVL(SUM(),0),2)[22], MIN()[22], MAX()[22]
   3 - (#keys=1; rowset=76) SUM()[22], "ITEM_6"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], NVL(SUM(),0)[22],
       ROUND(SUM()/NVL(SUM(),0),2)[22], MIN()[22], MAX()[22]
   4 - (rowset=76) SUM()[22], "ITEM_6"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], NVL(SUM(),0)[22],
       ROUND(SUM()/NVL(SUM(),0),2)[22], MIN()[22], MAX()[22]
   5 - (#keys=1) SUM()[22], "ITEM_6"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], NVL(SUM(),0)[22],
       ROUND(SUM()/NVL(SUM(),0),2)[22], MIN()[22], MAX()[22]
   6 - (#keys=2; rowset=76) "ITEM_6"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], SUM()[22], SUM()[22], SUM()[22],
       MAX()[22], MIN()[22]
   7 - (rowset=76) "ITEM_6"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], SYS_OP_MSR()[119]
   8 - (#keys=2) "ITEM_6"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], SYS_OP_MSR()[119]
   9 - (#keys=2; rowset=76) "ITEM_6"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], SYS_OP_MSR()[119]
  10 - (#keys=1; rowset=76) "ITEM_6"[VARCHAR2,800], "ITEM_2"[NUMBER,2], "ITEM_4"[NUMBER,2], "ITEM_5"[NUMBER,2],
       "ITEM_8"[VARCHAR2,200], "ITEM_2"[NUMBER,22], "ITEM_3"[NUMBER,22], "ITEM_4"[NUMBER,22], "ITEM_6"[NUMBER,22],
       "ITEM_7"[NUMBER,22]
  11 - (rowset=76) "ITEM_1"[NUMBER,22], "ITEM_2"[NUMBER,2], "ITEM_4"[NUMBER,2], "ITEM_5"[NUMBER,2],
       "ITEM_6"[VARCHAR2,800]
  12 - (rowset=76) "F"."FESTIVAL_ID"[NUMBER,22], "F"."FESTIVAL_NAME"[VARCHAR2,800]
  13 - (rowset=76) "F"."FESTIVAL_ID"[NUMBER,22], "F"."FESTIVAL_NAME"[VARCHAR2,800]
  14 - (rowset=76) "ITEM_1"[NUMBER,22], "ITEM_8"[VARCHAR2,200], "ITEM_2"[NUMBER,22], "ITEM_3"[NUMBER,22],
       "ITEM_4"[NUMBER,22], "ITEM_6"[NUMBER,22], "ITEM_7"[NUMBER,22]
  15 - (#keys=0) "ITEM_1"[NUMBER,22], "ITEM_8"[VARCHAR2,200], "ITEM_2"[NUMBER,22], "ITEM_3"[NUMBER,22],
       "ITEM_4"[NUMBER,22], "ITEM_6"[NUMBER,22], "ITEM_7"[NUMBER,22]
  16 - (rowset=76) "ITEM_1"[NUMBER,22], "ITEM_2"[NUMBER,22], "ITEM_3"[NUMBER,22], "ITEM_4"[NUMBER,22],
       "ITEM_6"[NUMBER,22], "ITEM_7"[NUMBER,22], "ITEM_8"[VARCHAR2,200]
  17 - (#keys=2; rowset=76) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], COUNT()[22], COUNT()[22],
       SUM()[22], MAX()[22], MIN()[22]
  18 - (rowset=256) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], SYS_OP_MSR()[89]
  19 - (#keys=2) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], SYS_OP_MSR()[89]
  20 - (#keys=2; rowset=256) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], SYS_OP_MSR()[89]
  21 - (rowset=256) "T"."TICKET_PRICE"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], "T"."FESTIVAL_ID"[NUMBER,22]
  22 - (rowset=256) "T"."TICKET_PRICE"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], "T"."FESTIVAL_ID"[NUMBER,22]
```

Notes

```
Note
-----
   - automatic DOP: Computed Degree of Parallelism is 2
```

2.2 Analyse van het plan Voor partitionering

Uit het explain plan Voor partitionering kunnen we de volgende vaststellingen doen:

- TABLE ACCESS FULL op TICKET (Id 22): Oracle scant de volledige TICKET-tabel van ~400 000 rijen. Er is geen partitionering aanwezig, dus de date-filter wordt pas Na het inlezen van alle rijen toegepast als een gewone predicate-filter op id 22.
- Interne groeperings-rewrite (VW_GBC_5 / VW_GBF_6): Oracle past een group-by view rewrite toe: TICKET wordt eerst geaggregeerd (ids 20–22) in de interne view VW_GBC_5, en daarna pas gejoind met FESTIVAL via VW_GBF_6. De kolomnamen in de column

projection verschijnen hierdoor als ITEM_1 t.e.m. ITEM_8 (positionale aliassen van de interne view).
- Hoge kost: de totale optimizer-kost bedraagt 1530. Het grootste deel (1523, ~99%) is toe te schrijven aan de volledige scan van TICKET (Id 21–22).
- Parallelle uitvoering (DOP=2): Oracle kiest automatisch voor parallelle uitvoering. Dit verklaart de PX-operatoren doorheen het plan. Zonder partitionering helpt parallelisme wel bij de uitvoeringssnelheid, maar lost het fundamentele scan-probleem niet op.
- Geschatte rijen TICKET: 181 000 rijen na de date-filter.
- Geschatte rijen resultatenset: 191 (werkelijk: 200). De optimizer onderschat het aantal groepen met slechts ~4.5%.

Stap 3: Optimalisatie + analyse

3.1 Gekozen techniek: RANGE-INTERVAL partitionering

De gekozen optimalisatietechniek is RANGE-INTERVAL partitionering op de kolom TICKET_PURCHASE_DATE. Deze keuze is om de volgende redenen logisch:

- De bedrijfsvraag filtert altijd op een afgebakend datumbereik (één kalenderjaar). Partitionering op de aankoopdatum zorgt ervoor dat Oracle enkel de relevante partitie(s) leest, alle andere worden genegeerd (partition pruning).
- INTERVAL-partitionering breidt RANGE uit met automatisch aangemaakte partities: zodra een rij met een aankoopdatum in een nog onbestaand jaar wordt ingevoegd, maakt Oracle zelfstandig een nieuwe jaarlijkse partitie aan. Dit garandeert toekomstbestendigheid zonder handmatige DDL-aanpassingen.
- De data verdeelt zich van nature per jaar (tickets worden gekocht in het jaar van het festival), waardoor de partities een gelijkmatige vulling krijgen en de pruning maximaal effectief is.

3.2 Partitie-script + uitleg partitie-sleutel

Stap 3A - Drop bestaande tabel

De bestaande niet-gepartitioneerde TICKET-tabel wordt verwijderd. CASCADE CONSTRAINTS zorgt ervoor dat ook de externe FK-verwijzingen (vanuit andere tabellen naar TICKET) automatisch worden verwijderd, zodat er geen resterende constraints achterblijven die het herscheppen zouden blokkeren.

```
DROP TABLE Ticket CASCADE CONSTRAINTS PURGE;
```

Stap 3B - Hercrëeer TICKET als gepartitioneerde tabel

De tabel wordt opnieuw aangemaakt met PARTITION BY RANGE (TICKET_PURCHASE_DATE) en INTERVAL (NUMTOYMINTERVAL(1, 'YEAR')). De structuur (kolommen, datatypes, constraints) is identiek aan de originele tabel.

```
CREATE TABLE Ticket
(
    Ticket_ID            NUMBER GENERATED ALWAYS AS IDENTITY,
    Ticket_Price         NUMBER(10, 2),
    Ticket_Currency      CHAR(3),
    Ticket_Date          DATE,
    Ticket_Purchase_Date DATE,
    Ticket_Description   VARCHAR2(100 CHAR),
    Ticket_Barcode       VARCHAR2(14 CHAR),
    Ticket_Type          VARCHAR2(50 CHAR),
    Ticket_Status        VARCHAR2(50 CHAR),
    Ticket_QR_Code       VARCHAR2(500 CHAR),
    TicketPlatform_ID    NUMBER,
    Visitor_ID           NUMBER,
    Festival_ID          NUMBER,
    CONSTRAINT Ticket_PK PRIMARY KEY (Ticket_ID),
    CONSTRAINT Ticket_Barcode_UQ UNIQUE (Ticket_Barcode),
    CONSTRAINT Ticket_VisitFestPlatform_UQ
        UNIQUE (Visitor_ID, Festival_ID, Ticket_Date),
    CONSTRAINT Ticket_Visitor_FK
        FOREIGN KEY (Visitor_ID) REFERENCES Visitor (Visitor_ID),
    CONSTRAINT Ticket_TicketPlatform_FK
        FOREIGN KEY (TicketPlatform_ID)
        REFERENCES TicketPlatform (TicketPlatform_ID),
    CONSTRAINT Ticket_Price_chk CHECK (Ticket_Price > 0),
    CONSTRAINT Ticket_Barcode_chk
        CHECK (LENGTH(Ticket_Barcode) BETWEEN 8 AND 14),
    CONSTRAINT Ticket_Date_chk
        CHECK (Ticket_Purchase_Date <= Ticket_Date)
)
    PARTITION BY RANGE (Ticket_Purchase_Date)
    INTERVAL (NUMTOYMINTERVAL(1, 'YEAR'))
(
    PARTITION ticket_p_pre2021 VALUES LESS THAN (DATE '2021-01-01'),
    PARTITION ticket_p_2021    VALUES LESS THAN (DATE '2022-01-01'),
    PARTITION ticket_p_2022    VALUES LESS THAN (DATE '2023-01-01'),
    PARTITION ticket_p_2023    VALUES LESS THAN (DATE '2024-01-01'),
    PARTITION ticket_p_2024    VALUES LESS THAN (DATE '2025-01-01'),
    PARTITION ticket_p_2025    VALUES LESS THAN (DATE '2026-01-01'),
    PARTITION ticket_p_2026    VALUES LESS THAN (DATE '2027-01-01')
);
```

Uitleg partitie-sleutel:

PARTITION BY RANGE (Ticket_Purchase_Date): de aankoopdatum bepaalt in welke partitie een rij terechtkomt. INTERVAL (NUMTOYMINTERVAL(1, 'YEAR')): elke nieuwe aankoopdatum buiten het laatste gedefinieerde bereik triggert automatisch een nieuwe jaarpartitie. Partities zijn LESS THAN gedefinieerd: ticket_p_2025 bevat alle rijen met Ticket_Purchase_Date >= 2025-01-01 EN < 2026-01-01.

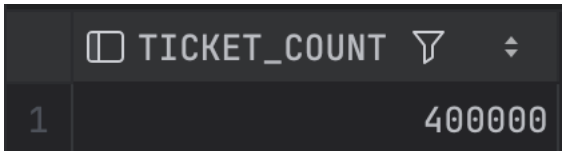
Stap 3C - Herstel Festival-FK

```
ALTER TABLE Ticket
    ADD CONSTRAINT Ticket_Festival_FK
        FOREIGN KEY (Festival_ID) REFERENCES Festival (Festival_ID);
```

Stap 3D - Data opnieuw genereren (minimum 400 000 tickets)

```
BEGIN
    DBMS_OUTPUT.ENABLE(NULL);
    PKG_S1_festivals.bewijs_milestone_6_S1(
        p_visitors        => 20000,
        p_companies       => 500,
        p_platforms       => 30,
        p_tickets         => 400000,
        p_permits_per_fest => 5,
        p_festivals       => 300,
        p_mode            => 'BULK'
    );
    COMMIT;
END;
/
```

Stap 3E - Rij-controle na hergeneratie



	TICKET_COUNT
1	400000

	FESTIVAL	TICKETTYPE	AANTAL_TICKETS	GEMIDDELDE_PRIJS_EUR	TOTALE_OMZET_EUR	LAAGSTE_PRIJS	HOOGSTE_PRIJS
1	Glastonbury Festival_0202_0000001102	Day	3666	275.98	1011730	50	
2	Glastonbury Festival_0202_0000001102	Full Festival	3601	278.85	1004136	50	
3	Glastonbury Festival_0202_0000001102	Weekend	3631	275.71	1001093	50	
4	Glastonbury Festival_0202_0000001102	Combi	3573	278.9	996520	50	
5	Glastonbury Festival_0202_0000001102	VIP	3548	270.14	958454	50	
6	Budapest Experience_0218_0000001118	Day	3352	278.63	933982	50	
7	Budapest Experience_0218_0000001118	Combi	3399	274.05	931486	50	
8	Budapest Experience_0218_0000001118	Weekend	3386	273.97	927654	50	
9	Budapest Experience_0218_0000001118	VIP	3302	276.36	912551	50	
10	Budapest Experience_0218_0000001118	Full Festival	3307	275.42	910822	50	
11	Budapest Open Air_0157_0000001057	Combi	3319	272.01	902815	50	
12	Budapest Open Air_0157_0000001057	VIP	3304	272.25	899529	50	
13	Budapest Open Air_0157_0000001057	Full Festival	3270	274.88	898842	50	
14	Budapest Open Air_0157_0000001057	Day	3237	276.7	895683	50	
15	Budapest Open Air_0157_0000001057	Weekend	3242	274.9	891234	50	
16	Zurich Open Air_0207_0000001107	Full Festival	2434	277.02	674259	50	
17	Zurich Open Air_0207_0000001107	Day	2438	274.57	669390	50	
18	Zurich Open Air_0207_0000001107	VIP	2350	276.25	649184	50	
19	Zurich Open Air_0207_0000001107	Weekend	2284	274.4	626739	50	
20	Zurich Open Air_0207_0000001107	Combi	2253	276.03	621886	50	
21	Paris Rave_0275_0000001175	Weekend	2088	277.48	579383	50	
22	Roskilde Festival_0043_0000000943	Full Festival	2019	277.2	559672	50	
23	Brussels Live_0229_0000001129	Combi	2006	275.87	553402	50	
24	Brussels Live_0229_0000001129	VIP	2010	275.28	553317	50	
25	Roskilde Festival_0043_0000000943	Combi	2033	271.92	552811	50	
26	Paris Rave_0275_0000001175	Full Festival	2011	274.62	552252	50	
27	Brussels Live_0229_0000001129	Day	1971	275.7	543397	50	
28	Roskilde Festival_0043_0000000943	Day	1939	278.63	540260	50	
29	Roskilde Festival_0043_0000000943	Weekend	2004	269.39	539854	50	
30	Paris Rave_0275_0000001175	Combi	1962	274.33	538227	50	
31	Brussels Live_0229_0000001129	Weekend	1908	281.24	536610	50	
32	Roskilde Festival_0043_0000000943	VIP	19	274.58	535985	50	

186 rows

Stap 3F - Statistieken herberekenen op de gepartitioneerde tabel

```
BEGIN
        DBMS_STATS.GATHER_TABLE_STATS('PROJECT', 'TICKET', granularity =>
        'ALL');
END;
```

/

Stap 3G - Grootte per partitie

Na de hergeneratie en statistieken-update kan de grootte per partitie worden gecontroleerd:

```
SELECT segment_name,
       segment_type,
       SUM(bytes / 1024 / 1024)      AS MB,
       (SELECT COUNT(*) FROM Ticket) AS table_count
FROM dba_segments
WHERE segment_name = 'TICKET'
GROUP BY segment_name, segment_type;

SELECT segment_name,
       partition_name,
       ROUND(bytes / 1024 / 1024, 2) AS MB
FROM dba_segments
WHERE segment_name = 'TICKET'
ORDER BY partition_name;

SELECT partition_name,
       partition_position          AS pos,
       num_rows,
       blocks,
       ROUND(blocks * 8 / 1024, 2) AS approx_MB
FROM   all_tab_partitions
WHERE  table_name = 'TICKET'
ORDER  BY partition_position;
```

	SEGMENT_NAME	SEGMENT_TYPE	MB	TABLE_COUNT
1	TICKET	TABLE PARTITION	88	400000

	SEGMENT_NAME	PARTITION_NAME	MB
1	TICKET	TICKET_P_2023	8
2	TICKET	TICKET_P_2024	32
3	TICKET	TICKET_P_2025	40
4	TICKET	TICKET_P_2026	8

	PARTITION_NAME	POS	NUM_ROWS	BLOCKS	APPROX_MB
1	TICKET_P_PRE2021	1	0	0	0
2	TICKET_P_2021	2	0	0	0
3	TICKET_P_2022	3	0	0	0
4	TICKET_P_2023	4	18975	1006	7.86
5	TICKET_P_2024	5	148149	4030	31.48
6	TICKET_P_2025	6	196366	5038	39.36
7	TICKET_P_2026	7	36510	1006	7.86

Opmerkingen bij de partitiegrootte:

- Partitie 6 (ticket_p_2025) bevat met 196 366 rijen het grootste deel van de data.
- Partities 1 t.e.m. 3 zijn volledig leeg (pre-2021, 2021, 2022). De generate random data functions maken alle tickets aan met aankoopdatums vanaf 2023, waardoor de eerste drie partities geen enkel record bevatten. Dit is een eigenschap van de testdata, niet van de partitioneringsstrategie zelf.
- De INTERVAL-clausule zorgt dat toekomstige invoegingen (bv. tickets voor 2027 en later) automatisch nieuwe partities genereren zonder DDL-aanpassingen.

Stap 3 (vervolg): Explain plans Na partitionering

3.3 Plan NA partitionering - met NOPARALLEL hint (Plan hash: 3331934735)

De NOPARALLEL-hint werd gebruikt zoals aanbevolen in de opdrachtbeschrijving (TIP3). Het dwingt seriële uitvoering af zodat het explain plan duidelijk aantoont welke individuele partitie bevraagd wordt, zonder dat de parallel-distributie-operatoren de leesbaarheid bemoeilijken.

```
EXPLAIN PLAN SET STATEMENT_ID = 'M7_S1_NA' FOR
SELECT /*+ NOPARALLEL(t) */
       f.Festival_Name               AS Festival,
       t.Ticket_Type                 AS Tickettype,
       COUNT(t.Ticket_ID)            AS Aantal_Tickets,
       ROUND(AVG(t.Ticket_Price), 2) AS Gemiddelde_Prijs_EUR,
       SUM(t.Ticket_Price)           AS Totale_Omzet_EUR,
       MIN(t.Ticket_Price)           AS Laagste_Prijs,
       MAX(t.Ticket_Price)           AS Hoogste_Prijs
FROM Ticket t
       JOIN Festival f ON f.Festival_ID = t.Festival_ID
WHERE t.Ticket_Purchase_Date
       BETWEEN DATE '2025-01-01' AND DATE '2025-12-31'
GROUP BY f.Festival_Name, t.Ticket_Type
ORDER BY Totale_Omzet_EUR DESC;
```

Execution plan

PLAN_TABLE_OUTPUT

```
Plan hash value: 3331934735

-----------------------------------------------------------------------------------------------------------------------------------------------
| Id  | Operation                          | Name     | Rows  | Bytes | Cost (%CPU)| Time     | Pstart| Pstop |    TQ  |IN-OUT| PQ Distrib |
-----------------------------------------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                   |          |  1058 | 63480 |  1379   (1)| 00:00:01 |       |       |        |      |            |
|   1 |  PX COORDINATOR                    |          |       |       |            |          |       |       |        |      |            |
|   2 |   PX SEND QC (ORDER)               | :TQ10003 |  1058 | 63480 |  1379   (1)| 00:00:01 |       |       | Q1,03 | P->S | QC (ORDER) |
|   3 |    SORT ORDER BY                   |          |  1058 | 63480 |  1379   (1)| 00:00:01 |       |       | Q1,03 | PCWP |            |
|   4 |     PX RECEIVE                     |          |  1058 | 63480 |  1379   (1)| 00:00:01 |       |       | Q1,03 | PCWP |            |
|   5 |      PX SEND RANGE                 | :TQ10002 |  1058 | 63480 |  1379   (1)| 00:00:01 |       |       | Q1,02 | P->P | RANGE      |
|   6 |       HASH GROUP BY                |          |  1058 | 63480 |  1379   (1)| 00:00:01 |       |       | Q1,02 | PCWP |            |
|   7 |        PX RECEIVE                  |          |  1058 | 63480 |  1379   (1)| 00:00:01 |       |       | Q1,02 | PCWP |            |
|   8 |         PX SEND HASH               | :TQ10001 |  1058 | 63480 |  1379   (1)| 00:00:01 |       |       | Q1,01 | P->P | HASH       |
|   9 |          HASH GROUP BY             |          |  1058 | 63480 |  1379   (1)| 00:00:01 |       |       | Q1,01 | PCWP |            |
|* 10 |           HASH JOIN                |          |   196K|    11M|  1372   (1)| 00:00:01 |       |       | Q1,01 | PCWP |            |
|  11 |            PX BLOCK ITERATOR       |          |   300 | 10800 |     2   (0)| 00:00:01 |       |       | Q1,01 | PCWC |            |
|  12 |             TABLE ACCESS FULL      | FESTIVAL |   300 | 10800 |     2   (0)| 00:00:01 |       |       | Q1,01 | PCWP |            |
|  13 |            BUFFER SORT             |          |       |       |            |          |       |       | Q1,01 | PCWC |            |
|  14 |             PX RECEIVE             |          |   196K|  4600K|  1370   (1)| 00:00:01 |       |       | Q1,01 | PCWP |            |
|  15 |              PX SEND BROADCAST     | :TQ10000 |   196K|  4600K|  1370   (1)| 00:00:01 |       |       |        | S->P | BROADCAST  |
|  16 |               PARTITION RANGE SINGLE|          |   196K|  4600K|  1370   (1)| 00:00:01 |     6 |     6 |        |      |            |
|* 17 |                TABLE ACCESS FULL   | TICKET   |   196K|  4600K|  1370   (1)| 00:00:01 |     6 |     6 |        |      |            |
-----------------------------------------------------------------------------------------------------------------------------------------------

Query Block Name / Object Alias (identified by operation id):
-------------------------------------------------------------

   1 - SEL$58A6D7F6
  12 - SEL$58A6D7F6 / F@SEL$1
  17 - SEL$58A6D7F6 / T@SEL$1
```

76 rows

Sleutelbewijs partitionering: Id 16 toont PARTITION RANGE SINGLE met Pstart=6, Pstop=6. Oracle leest uitsluitend partitie 6 (ticket_p_2025). De overige 6 partities worden volledig genegeerd. Dit is partition pruning in werking.

Hint Report: NOPARALLEL(t) is Unused (U) op Id 17 – de hint werd herkend maar de TICKET-scan op Id 15-17 loopt al serieel (S→P) door de forced serial mode.

Predicate Information

```
Predicate Information (identified by operation id):
---------------------------------------------------

  10 - access("F"."FESTIVAL_ID"="T"."FESTIVAL_ID")
  17 - filter("T"."TICKET_PURCHASE_DATE"<=TO_DATE(' 2025-12-31 00:00:00', 'syyyy-mm-dd hh24:mi:ss') AND
              "T"."TICKET_PURCHASE_DATE">=TO_DATE(' 2025-01-01 00:00:00', 'syyyy-mm-dd hh24:mi:ss'))
```

Column Projection Information

```
Column Projection Information (identified by operation id):
-----------------------------------------------------------

   1 - SUM()[22], "F"."FESTIVAL_NAME"[VARCHAR2,800], "T"."TICKET_TYPE"[VARCHAR2,200], COUNT()[22], ROUND(SUM()/COUNT(),2)[22],
       MIN()[22], MAX()[22]
   2 - (#keys=0) SUM()[22], "F"."FESTIVAL_NAME"[VARCHAR2,800], "T"."TICKET_TYPE"[VARCHAR2,200], COUNT()[22],
       ROUND(SUM()/COUNT(),2)[22], MIN()[22], MAX()[22]
   3 - (#keys=1; rowset=76) SUM()[22], "F"."FESTIVAL_NAME"[VARCHAR2,800], "T"."TICKET_TYPE"[VARCHAR2,200], COUNT()[22],
       ROUND(SUM()/COUNT(),2)[22], MIN()[22], MAX()[22]
   4 - (rowset=76) SUM()[22], "F"."FESTIVAL_NAME"[VARCHAR2,800], "T"."TICKET_TYPE"[VARCHAR2,200], COUNT()[22],
       ROUND(SUM()/COUNT(),2)[22], MIN()[22], MAX()[22]
   5 - (#keys=1) SUM()[22], "F"."FESTIVAL_NAME"[VARCHAR2,800], "T"."TICKET_TYPE"[VARCHAR2,200], COUNT()[22],
       ROUND(SUM()/COUNT(),2)[22], MIN()[22], MAX()[22]
   6 - (#keys=2; rowset=76) "F"."FESTIVAL_NAME"[VARCHAR2,800], "T"."TICKET_TYPE"[VARCHAR2,200], COUNT()[22], COUNT()[22], SUM()[22],
       MAX()[22], MIN()[22]
   7 - (rowset=76) "F"."FESTIVAL_NAME"[VARCHAR2,800], "T"."TICKET_TYPE"[VARCHAR2,200], SYS_OP_MSR()[89]
   8 - (#keys=2) "F"."FESTIVAL_NAME"[VARCHAR2,800], "T"."TICKET_TYPE"[VARCHAR2,200], SYS_OP_MSR()[89]
   9 - (#keys=2; rowset=76) "F"."FESTIVAL_NAME"[VARCHAR2,800], "T"."TICKET_TYPE"[VARCHAR2,200], SYS_OP_MSR()[89]
  10 - (#keys=1; rowset=76) "F"."FESTIVAL_NAME"[VARCHAR2,800], "T"."TICKET_PRICE"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200]
  11 - (rowset=76) "F"."FESTIVAL_ID"[NUMBER,22], "F"."FESTIVAL_NAME"[VARCHAR2,800]
  12 - (rowset=76) "F"."FESTIVAL_ID"[NUMBER,22], "F"."FESTIVAL_NAME"[VARCHAR2,800]
  13 - (#keys=0; rowset=76) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_PRICE"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200]
  14 - (rowset=256) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_PRICE"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200]
  15 - (#keys=0) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_PRICE"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200]
  16 - (rowset=256) "T"."TICKET_PRICE"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], "T"."FESTIVAL_ID"[NUMBER,22]
  17 - (rowset=256) "T"."TICKET_PRICE"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], "T"."FESTIVAL_ID"[NUMBER,22]

Hint Report (identified by operation id / Query Block Name / Object Alias):
Total hints for statement: 1 (U - Unused (1))
---------------------------------------------------------------------------

  17 -  SEL$58A6D7F6 / T@SEL$1
         U -  NOPARALLEL(t)

Note
-----
   - automatic DOP: Computed Degree of Parallelism is 2
```

Explain plan NA partitionering - met NOPARALLEL hint (Plan hash: 3331934735)

Uit het explain plan Na partitionering met de NOPARALLEL-hint kunnen we de volgende vaststellingen doen:

- PARTITION RANGE SINGLE (Id 16): Oracle leest uitsluitend partitie 6 (ticket_p_2025) aan, zoals bevestigd door Pstart=6 en Pstop=6. De overige zes partities worden volledig genegeerd. Dit is partition pruning in werking, de voornaamste doelstelling van de optimalisatie.
- Kostenverlaging: de totale kost bedraagt 1379, tegenover 1530 Voor de partitionering (~10% reductie).
- Seriële TICKET-scan (S→P op Id 15): De NOPARALLEL-hint dwingt de TICKET-scan serieel af. Oracle compenseert dit met een BUFFER SORT (Id 13) en PX SEND BROADCAST (Id 15) om de serieel gelezen data te distribueren naar de parallelle slaves voor de join met FESTIVAL.
- Geschatte rijen TICKET: 196K rijen, dit stemt nu veel beter overeen met de werkelijke partitiegrootte (196.366 rijen in ticket_p_2025), wat bevestigt dat de partitie-niveau statistieken correct zijn berekend.

- Hint Report: NOPARALLEL(t) staat als Unused (U) op Id 17, de hint werd herkend maar de TICKET-scan loopt al serieel door de geforceerde modus.

3.4 Plan NA partitionering – volledig parallel, zonder hint (Plan hash: 2136075909)

Zonder de NOPARALLEL-hint maakt Oracle optimaal gebruik van zowel partitionering als parallelle uitvoering. Dit geeft het meest performante plan.

Execution plan

PLAN_TABLE_OUTPUT

```
Plan hash value: 2136075909

-----------------------------------------------------------------------------------------------------------------------------------------------
| Id  | Operation                          | Name     | Rows  | Bytes | Cost (%CPU)| Time     | Pstart| Pstop |    TQ  |IN-OUT| PQ Distrib |
-----------------------------------------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                   |          |   135 | 15255 |   768   (1)| 00:00:01 |       |       |        |      |            |
|   1 |  PX COORDINATOR                    |          |       |       |            |          |       |       |        |      |            |
|   2 |   PX SEND QC (ORDER)               | :TQ10004 |   135 | 15255 |   768   (1)| 00:00:01 |       |       |  Q1,04 | P->S | QC (ORDER) |
|   3 |    SORT ORDER BY                   |          |   135 | 15255 |   768   (1)| 00:00:01 |       |       |  Q1,04 | PCWP |            |
|   4 |     PX RECEIVE                     |          |   135 | 15255 |   768   (1)| 00:00:01 |       |       |  Q1,04 | PCWP |            |
|   5 |      PX SEND RANGE                 | :TQ10003 |   135 | 15255 |   768   (1)| 00:00:01 |       |       |  Q1,03 | P->P | RANGE      |
|   6 |       HASH GROUP BY                |          |   135 | 15255 |   768   (1)| 00:00:01 |       |       |  Q1,03 | PCWP |            |
|   7 |        PX RECEIVE                  |          |   135 | 15255 |   768   (1)| 00:00:01 |       |       |  Q1,03 | PCWP |            |
|   8 |         PX SEND HASH               | :TQ10002 |   135 | 15255 |   768   (1)| 00:00:01 |       |       |  Q1,02 | P->P | HASH       |
|   9 |          HASH GROUP BY             |          |   135 | 15255 |   768   (1)| 00:00:01 |       |       |  Q1,02 | PCWP |            |
|* 10 |           HASH JOIN                |          |   135 | 15255 |   766   (1)| 00:00:01 |       |       |  Q1,02 | PCWP |            |
|  11 |            PX BLOCK ITERATOR       |          |   300 | 10800 |     2   (0)| 00:00:01 |       |       |  Q1,02 | PCWC |            |
|  12 |             TABLE ACCESS FULL      | FESTIVAL |   300 | 10800 |     2   (0)| 00:00:01 |       |       |  Q1,02 | PCWP |            |
|  13 |            PX RECEIVE              |          |   135 | 10395 |   764   (1)| 00:00:01 |       |       |  Q1,02 | PCWP |            |
|  14 |             PX SEND BROADCAST      | :TQ10001 |   135 | 10395 |   764   (1)| 00:00:01 |       |       |  Q1,01 | P->P | BROADCAST  |
|  15 |              VIEW                  | VW_GBC_5 |   135 | 10395 |   764   (1)| 00:00:01 |       |       |  Q1,01 | PCWP |            |
|  16 |               HASH GROUP BY        |          |   135 |  3240 |   764   (1)| 00:00:01 |       |       |  Q1,01 | PCWP |            |
|  17 |                PX RECEIVE          |          |   135 |  3240 |   764   (1)| 00:00:01 |       |       |  Q1,01 | PCWP |            |
|  18 |                 PX SEND HASH       | :TQ10000 |   135 |  3240 |   764   (1)| 00:00:01 |       |       |  Q1,00 | P->P | HASH       |
|  19 |                  HASH GROUP BY     |          |   135 |  3240 |   764   (1)| 00:00:01 |       |       |  Q1,00 | PCWP |            |
|  20 |                   PX BLOCK ITERATOR|          |   196K|  4600K|   760   (1)| 00:00:01 |     6 |     6 |  Q1,00 | PCWC |            |
|* 21 |                    TABLE ACCESS FULL| TICKET   |   196K|  4600K|   760   (1)| 00:00:01 |     6 |     6 |  Q1,00 | PCWP |            |
-----------------------------------------------------------------------------------------------------------------------------------------------

Query Block Name / Object Alias (identified by operation id):
-------------------------------------------------------------

   1 - SEL$36E7E959
  12 - SEL$36E7E959 / F@SEL$1
  15 - SEL$8B9A55F1 / VW_GBC_5@SEL$6C2C6143
  16 - SEL$8B9A55F1
  21 - SEL$8B9A55F1 / T@SEL$1
```

Predicate Information

```
Predicate Information (identified by operation id):
---------------------------------------------------

  10 - access("F"."FESTIVAL_ID"="ITEM_1")
  21 - filter("T"."TICKET_PURCHASE_DATE"<=TO_DATE(' 2025-12-31 00:00:00', 'syyyy-mm-dd hh24:mi:ss') AND
              "T"."TICKET_PURCHASE_DATE">=TO_DATE(' 2025-01-01 00:00:00', 'syyyy-mm-dd hh24:mi:ss'))
```

Column Projection Information

```
Column Projection Information (identified by operation id):
-----------------------------------------------------------

   1 - SUM()[22], "F"."FESTIVAL_NAME"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], NVL(SUM(),0)[22], ROUND(SUM()/NVL(SUM(),0),2)[22],
       MIN()[22], MAX()[22]
   2 - (#keys=0) SUM()[22], "F"."FESTIVAL_NAME"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], NVL(SUM(),0)[22],
       ROUND(SUM()/NVL(SUM(),0),2)[22], MIN()[22], MAX()[22]
   3 - (#keys=1; rowset=76) SUM()[22], "F"."FESTIVAL_NAME"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], NVL(SUM(),0)[22],
       ROUND(SUM()/NVL(SUM(),0),2)[22], MIN()[22], MAX()[22]
   4 - (rowset=76) SUM()[22], "F"."FESTIVAL_NAME"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], NVL(SUM(),0)[22],
       ROUND(SUM()/NVL(SUM(),0),2)[22], MIN()[22], MAX()[22]
   5 - (#keys=1) SUM()[22], "F"."FESTIVAL_NAME"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], NVL(SUM(),0)[22],
       ROUND(SUM()/NVL(SUM(),0),2)[22], MIN()[22], MAX()[22]
   6 - (#keys=2; rowset=76) "F"."FESTIVAL_NAME"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], SUM()[22], SUM()[22], SUM()[22], MAX()[22],
       MIN()[22]
   7 - (rowset=76) "F"."FESTIVAL_NAME"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], SYS_OP_MSR()[119]
   8 - (#keys=2) "F"."FESTIVAL_NAME"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], SYS_OP_MSR()[119]
   9 - (#keys=2; rowset=76) "F"."FESTIVAL_NAME"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], SYS_OP_MSR()[119]
  10 - (#keys=1; rowset=76) "F"."FESTIVAL_NAME"[VARCHAR2,800], "ITEM_8"[VARCHAR2,200], "ITEM_2"[NUMBER,22], "ITEM_3"[NUMBER,22],
       "ITEM_4"[NUMBER,22], "ITEM_6"[NUMBER,22], "ITEM_7"[NUMBER,22]
  11 - (rowset=76) "F"."FESTIVAL_ID"[NUMBER,22], "F"."FESTIVAL_NAME"[VARCHAR2,800]
  12 - (rowset=76) "F"."FESTIVAL_ID"[NUMBER,22], "F"."FESTIVAL_NAME"[VARCHAR2,800]
  13 - (rowset=76) "ITEM_1"[NUMBER,22], "ITEM_8"[VARCHAR2,200], "ITEM_2"[NUMBER,22], "ITEM_3"[NUMBER,22], "ITEM_4"[NUMBER,22],
       "ITEM_6"[NUMBER,22], "ITEM_7"[NUMBER,22]
  14 - (#keys=0) "ITEM_1"[NUMBER,22], "ITEM_8"[VARCHAR2,200], "ITEM_2"[NUMBER,22], "ITEM_3"[NUMBER,22], "ITEM_4"[NUMBER,22],
       "ITEM_6"[NUMBER,22], "ITEM_7"[NUMBER,22]
  15 - (rowset=76) "ITEM_1"[NUMBER,22], "ITEM_2"[NUMBER,22], "ITEM_3"[NUMBER,22], "ITEM_4"[NUMBER,22], "ITEM_6"[NUMBER,22],
       "ITEM_7"[NUMBER,22], "ITEM_8"[VARCHAR2,200]
  16 - (#keys=2; rowset=76) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], COUNT()[22], COUNT()[22], SUM()[22],
       MAX()[22], MIN()[22]
```

```
  17 - (rowset=256) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], SYS_OP_MSR()[89]
  18 - (#keys=2) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], SYS_OP_MSR()[89]
  19 - (#keys=2; rowset=256) "T"."FESTIVAL_ID"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], SYS_OP_MSR()[89]
  20 - (rowset=256) "T"."TICKET_PRICE"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], "T"."FESTIVAL_ID"[NUMBER,22]
  21 - (rowset=256) "T"."TICKET_PRICE"[NUMBER,22], "T"."TICKET_TYPE"[VARCHAR2,200], "T"."FESTIVAL_ID"[NUMBER,22]

Note
-----
   - automatic DOP: Computed Degree of Parallelism is 2
   - This is SQL Plan Management Test Plan
```

Plan NA partitionering - volledig parallel, zonder hint (Plan hash: 2136075909)

- Partition pruning actief: Id 20–21 tonen PX BLOCK ITERATOR + TABLE ACCESS FULL op TICKET met Pstart=6, Pstop=6. Oracle leest uitsluitend partitie 6 parallel, identiek aan het vorige parallelle plan.

- Group-by rewrite (VW_GBC_5): Net als bij het Voor-plan kiest de optimizer opnieuw voor een interne group-by view rewrite. TICKET wordt eerst geaggregeerd (ids 19–21) in VW_GBC_5 Voor de join met FESTIVAL. De kolomaliassen verschijnen als ITEM_1 t.e.m. ITEM_8 in de column projection.
- Kostenverlaging: totale kost bedraagt 768, tegenover 1379 voor het NOPARALLEL-plan (~44% reductie) en 1530 Voor de partitionering (~50% reductie). Dit is de beste kost van alle drie de scenario's.
- Geschatte rijen resultatenset: 135 (werkelijk: 176). De optimizer onderschat nu het aantal groepen licht, tegenover een overschatting van 191 in het NOPARALLEL-plan.

Stap 4: Vergelijking Voor / Na optimalisatie

4.1 Kostenvergelijking

Scenario	Plan hash	Total cost	TICKET scan cost	Partitie scan?	Parallel?
Voor - geen partities, parallel	2956872105	1530	1523 (99%)	Nee - volledige tabel	Ja (DOP=2)
Na - partities + NOPARALLEL hint	3331934735	1379	1370 (99%)	Ja - partitie 6 only	Serieel (S→P)
Na - partities + volledig parallel	2136075909	768	760 (99%)	Ja - partitie 6 only	Ja (DOP=2)

4.2 Kostenreductie

Vergelijking	Kostenbesparing	Verbetering
Voor (1530) => Na + NOPARALLEL (1379)	151 kostpunten	~10%
Voor (1530) => Na + parallel (768)	762 kostpunten	~50%
NOPARALLEL (1379) => Parallel (768)	611 kostpunten	~44%

4.3 Toelichting rijen-schattingen

De geschatte rijen voor de TICKET-scan bedragen 196K in beide Na-plannen, wat nauwkeurig overeenkomt met de werkelijke partitiegrootte (196 366 rijen). De geschatte resultatenset daalt van 191 (Voor) naar 135 (Na), terwijl het werkelijke aantal 176 rijen bedraagt, de optimizer onderschat nu licht (~23%), tegenover een overschatting van ~8% Voor de partitionering. Dit is een gevolg van de verfijnde partitie-statistieken.

Stap 5: Conclusie S1 - Romeo Weyns

De implementatie van RANGE-INTERVAL partitionering op TICKET_PURCHASE_DATE levert een duidelijke en meetbare performantieverbetering voor de domeinspecifieke aggregatiequery.

Het centrale mechanisme is partition pruning: doordat de WHERE-clausule een exact datumbereik van een kalenderjaar opgeeft, kan Oracle de Pstart- en Pstop-waarden statisch bepalen (beide = 6) en alle andere partities volledig overslaan. In de explain plannen is dit zichtbaar als PARTITION RANGE SINGLE voor het seriële plan en als Pstart=6/Pstop=6 bij de PX BLOCK ITERATOR voor het parallelle plan.

- Partitionering alleen (NOPARALLEL): kostenreductie van ~10% (1530 => 1379). De winst is beperkt omdat de TICKET-scan serieel blijft draaien, wat extra distributiestappen vereist (BUFFER SORT, BROADCAST). Oracle compenseert de gemengde serieel/parallel-omgeving met een complexer distributieplan.
- Partitionering + parallelle uitvoering: kostenreductie van ~50% (1530 => 768). Door de NOPARALLEL-hint te verwijderen, scant Oracle partitie 6 parallel via PX BLOCK ITERATOR. Dit is het meest efficiënte plan: partition pruning elimineert onnodige I/O, en parallelle uitvoering maximaliseert de doorvoersnelheid op de benodigde data.
- INTERVAL-partitionering als toekomstgarantie: de INTERVAL-clausule zorgt ervoor dat bij het invoegen van tickets met een aankoopdatum in 2027 of later automatisch nieuwe jaarpartities worden aangemaakt. Er zijn geen handmatige DDL-aanpassingen nodig, wat de onderhoudbaarheid ten goede komt.
- Rijen-schattingen: de optimizer onderschat de eindresultatenset licht (135 geschat vs werkelijk 176, ~23%). Dit is een omgekeerde tendens tegenover het voor-plan, waar de resultatenset overschat werd (191 geschat). De verfijnde partitie-statistieken na GATHER_TABLE_STATS met granularity => 'ALL' hebben de schatting op scanniveau sterk verbeterd (196K geschat vs 196.366 werkelijk), maar de schatting van het aantal unieke combinaties van Festival_Name x Ticket_Type blijft een bron van onnauwkeurigheid. Extended statistics op deze combinatie zouden de schattingen verder verbeteren, al heeft dit geen invloed op het partition-pruning-gedrag.

Samengevat geldt: partitionering op een datumkolom is de meest logische en impactvolle optimalisatietechniek wanneer queries consistent filteren op een datumbereik dat een beperkt deel van de tabel omvat. De combinatie met Oracle's automatische parallelle uitvoering (DOP=2) versterkt dit effect aanzienlijk en levert een plan dat zowel I/O- als CPU-efficiënt is. De aanwezigheid van lege partities (2021-2022) heeft geen invloed op de query-performantie, Oracle slaat deze over zonder extra kost.

S2 – Jonas Friedhoff

Stap 1: Voorbereiding

Tabelinfo en grootte van de brontabel

De brontabel D/Z voor S2 is Performance, met 559 240 rijen die ~144 MB aan disk-ruimte innemen. Dit voldoet ruim aan de minimum-eisen van 400 000 rijen en 60 MB.

	SEGMENT_NAME	SEGMENT_TYPE	MB	TABLE_COUNT
1	PERFORMANCE	TABLE	144	559240

Rij-aantallen per tabel

De aggregate query joint vier tabellen: Performance (de grote D/Z-tabel), Stage, Festival en Artist.

	TAB	N
1	PERFORMANCE	559240
2	STAGE	9320
3	FESTIVAL	310
4	ARTIST	310
5	GENRE	40

Domeinspecifieke geaggregeerde query

De query produceert een "festival programming dashboard": per festival het totaal aantal performances, de totale stage-tijd in minuten en het totale artiest-budget. De aggregatie reduceert 559 240 Performance-rijen tot 310 festival-rijen (~1800× reductie in resultatenset).

De query is FAST-refresh compatibel geschreven (paired COUNT(expr) voor elke SUM(expr), geen COUNT(DISTINCT)) zodat de materialized view rechtstreeks vanuit deze query gedefinieerd kan worden.

```
SELECT
    f.Festival_ID,
    f.Festival_Name,
    COUNT(*)                                                    AS
Performance_Count,
    SUM((p.Performance_End_Time - p.Performance_Start_Time) * 24 * 60)  AS Total_Minutes,
    COUNT((p.Performance_End_Time - p.Performance_Start_Time) * 24 * 60) AS Cnt_Minutes,
    SUM(a.Artist_Price)                                         AS
Total_Artist_Spend,
    COUNT(a.Artist_Price)                                       AS Cnt_Artist_Price
FROM Performance p
JOIN Stage    s ON s.Stage_ID    = p.Stage_ID
JOIN Festival f ON f.Festival_ID = s.Festival_ID
JOIN Artist   a ON a.Artist_ID   = p.Artist_ID
GROUP BY f.Festival_ID, f.Festival_Name;
```

Stap 2: Analyse Voor optimalisatie

Explain plan Voor optimalisatie

Het plan toont vier full table scans (Festival, Artist, Stage, Performance), drie hash joins en een hash group by. Oracle parallelliseert de uitvoering over meerdere PX-slaves. Totale kost: 2860.

```
Plan hash value: 81020357

-------------------------------------------------------------------------------------------------------------------
| Id  | Operation                  | Name        | Rows  | Bytes | Cost (%CPU)| Time     |    TQ  |IN-OUT| PQ Distrib |
-------------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT           |             |   310 | 33480 |  2860   (1)| 00:00:01 |        |      |            |
|   1 |  PX COORDINATOR            |             |       |       |            |          |        |      |            |
|   2 |   PX SEND QC (RANDOM)      | :TQ10002    |   310 | 33480 |  2860   (1)| 00:00:01 |  Q1,02 | P→S  | QC (RAND)  |
|*  3 |    HASH JOIN BUFFERED      |             |   310 | 33480 |  2860   (1)| 00:00:01 |  Q1,02 | PCWP |            |
|   4 |     PX BLOCK ITERATOR      |             |   310 | 17360 |     3   (0)| 00:00:01 |  Q1,02 | PCWC |            |
|   5 |      TABLE ACCESS FULL     | FESTIVAL    |   310 | 17360 |     3   (0)| 00:00:01 |  Q1,02 | PCWP |            |
|   6 |     PX RECEIVE             |             |   311 | 16172 |  2857   (1)| 00:00:01 |  Q1,02 | PCWP |            |
|   7 |      PX SEND BROADCAST     | :TQ10001    |   311 | 16172 |  2857   (1)| 00:00:01 |  Q1,01 | P→P  | BROADCAST  |
|   8 |       VIEW                 | VW_GBC_10   |   311 | 16172 |  2857   (1)| 00:00:01 |  Q1,01 | PCWP |            |
|   9 |        HASH GROUP BY       |             |   311 | 12751 |  2857   (1)| 00:00:01 |  Q1,01 | PCWP |            |
|  10 |         PX RECEIVE         |             |   311 | 12751 |  2857   (1)| 00:00:01 |  Q1,01 | PCWP |            |
|  11 |          PX SEND HASH      | :TQ10000    |   311 | 12751 |  2857   (1)| 00:00:01 |  Q1,00 | P→P  | HASH       |
|  12 |           HASH GROUP BY    |             |   311 | 12751 |  2857   (1)| 00:00:01 |  Q1,00 | PCWP |            |
|* 13 |            HASH JOIN       |             |   555K|    21M|  2848   (1)| 00:00:01 |  Q1,00 | PCWP |            |
|  14 |             TABLE ACCESS FULL | ARTIST    |   310 |  2790 |     2   (0)| 00:00:01 |  Q1,00 | PCWP |            |
|* 15 |             HASH JOIN      |             |   551K|    16M|  2846   (1)| 00:00:01 |  Q1,00 | PCWP |            |
|  16 |              TABLE ACCESS FULL | STAGE   |  9320 | 74560 |    94   (0)| 00:00:01 |  Q1,00 | PCWP |            |
|  17 |              PX BLOCK ITERATOR |         |   559K|    12M|  2750   (1)| 00:00:01 |  Q1,00 | PCWC |            |
|  18 |               TABLE ACCESS FULL| PERFORMANCE | 559K|    12M|  2750   (1)| 00:00:01 |  Q1,00 | PCWP |            |
-------------------------------------------------------------------------------------------------------------------
```

Wall-clock tijd Voor optimalisatie

Met query rewrite uitgeschakeld levert de query 310 rijen in:

```
Cold cache : 550 ms execution
Hot cache : 105 ms execution
```

Stap 3: Optimalisatie + analyse

Gekozen techniek: Materialized View met FAST ON COMMIT en QUERY REWRITE

Een materialized view (MV) precomputeert het aggregaat één keer en bewaart het als fysieke tabel. Drie keuzes maken deze MV bijzonder krachtig:

• REFRESH FAST: de MV wordt incrementeel bijgewerkt via materialized view logs op de bron-tabellen, alleen de wijzigingen ("deltas") worden toegepast, geen volledige hercomputatie.

• ON COMMIT: bij elke COMMIT op een bron-tabel wordt de MV binnen dezelfde transactie meegerefresht. De MV is dus altijd transactie-fris.

• ENABLE QUERY REWRITE: de optimizer mag de originele query automatisch naar de MV omleiden, geen aanpassing aan applicatiecode nodig.

Materialized view logs (refresh-mechanisme: deel 1)

Voor REFRESH FAST heeft elke bron-tabel een MV-log nodig die wijzigingen registreert. De clausules zijn niet vrijblijvend:

• WITH ROWID, verplicht voor join-MVs zodat Oracle de gewijzigde rij kan terugvinden in de MV.

• SEQUENCE, bewaart de volgorde van wijzigingen, nodig voor correcte FAST-refresh bij meerdere DML-operaties op dezelfde rij.

• INCLUDING NEW VALUES, verplicht voor aggregate-MVs: zonder dit kan Oracle SUM- en COUNT-deltas niet bovenop bestaande aggregaten optellen.

```
-- ========================================================================
-- S2 - Milestone 7 - Materialized View Logs
-- Run order: 4
-- ========================================================================
-- Required for REFRESH FAST ON COMMIT.
-- Each base table referenced by the MV needs a log that captures:
--   * ROWID                  (mandatory for join MVs)
--   * INCLUDING NEW VALUES   (mandatory for aggregate MVs)
--   * Every column used by the MV in its filter, group-by and aggregates
-- ========================================================================

DROP MATERIALIZED VIEW LOG ON Performance;
DROP MATERIALIZED VIEW LOG ON Stage;
DROP MATERIALIZED VIEW LOG ON Festival;
DROP MATERIALIZED VIEW LOG ON Artist;

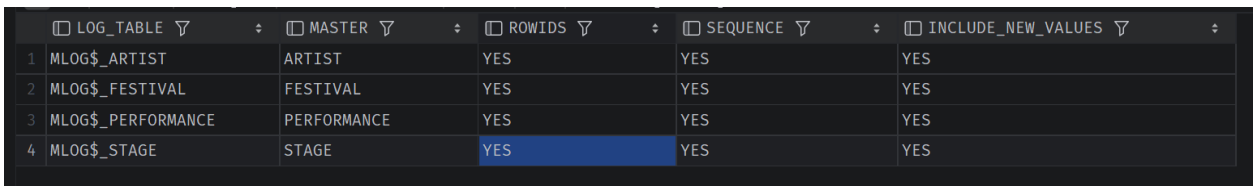
CREATE MATERIALIZED VIEW LOG ON Performance
    WITH ROWID, SEQUENCE
    (Performance_Start_Time, Performance_End_Time, Stage_ID, Artist_ID)
    INCLUDING NEW VALUES;

CREATE MATERIALIZED VIEW LOG ON Stage
    WITH ROWID, SEQUENCE
    (Stage_ID, Festival_ID)
    INCLUDING NEW VALUES;

CREATE MATERIALIZED VIEW LOG ON Festival
    WITH ROWID, SEQUENCE
    (Festival_ID, Festival_Name)
    INCLUDING NEW VALUES;

CREATE MATERIALIZED VIEW LOG ON Artist
    WITH ROWID, SEQUENCE
    (Artist_ID, Artist_Price)
    INCLUDING NEW VALUES;
```

Verificatie van de log-configuratie:



	LOG_TABLE	MASTER	ROWIDS	SEQUENCE	INCLUDE_NEW_VALUES
1	MLOG$_ARTIST	ARTIST	YES	YES	YES
2	MLOG$_FESTIVAL	FESTIVAL	YES	YES	YES
3	MLOG$_PERFORMANCE	PERFORMANCE	YES	YES	YES
4	MLOG$_STAGE	STAGE	YES	YES	YES

Materialized view definitie (refresh-mechanisme: deel 2)

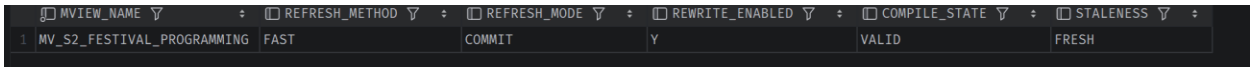
De MV wordt direct opgebouwd (BUILD IMMEDIATE), refresht zichzelf bij elke COMMIT op de bron-tabellen, en is beschikbaar voor query rewrite.

```
DROP MATERIALIZED VIEW MV_S2_Festival_Programming;

CREATE MATERIALIZED VIEW MV_S2_Festival_Programming
    BUILD IMMEDIATE
    REFRESH FAST ON COMMIT
    ENABLE QUERY REWRITE
AS
SELECT
    f.Festival_ID,
    f.Festival_Name,
    COUNT(*)                                                          AS
Performance_Count,
    SUM((p.Performance_End_Time - p.Performance_Start_Time) * 24 * 60)  AS Total_Minutes,
    COUNT((p.Performance_End_Time - p.Performance_Start_Time) * 24 * 60) AS Cnt_Minutes,
    SUM(a.Artist_Price)                                               AS
Total_Artist_Spend,
    COUNT(a.Artist_Price)                                             AS Cnt_Artist_Price
FROM Performance p
JOIN Stage    s ON s.Stage_ID    = p.Stage_ID
JOIN Festival f ON f.Festival_ID = s.Festival_ID
JOIN Artist   a ON a.Artist_ID   = p.Artist_ID
GROUP BY f.Festival_ID, f.Festival_Name;

-- Verify the MV ended up in the intended state
SELECT mview_name, refresh_method, refresh_mode, rewrite_enabled, compile_state, staleness
FROM   user_mviews
WHERE  mview_name = 'MV_S2_FESTIVAL_PROGRAMMING';
```

Verificatie van de MV-status na creatie:

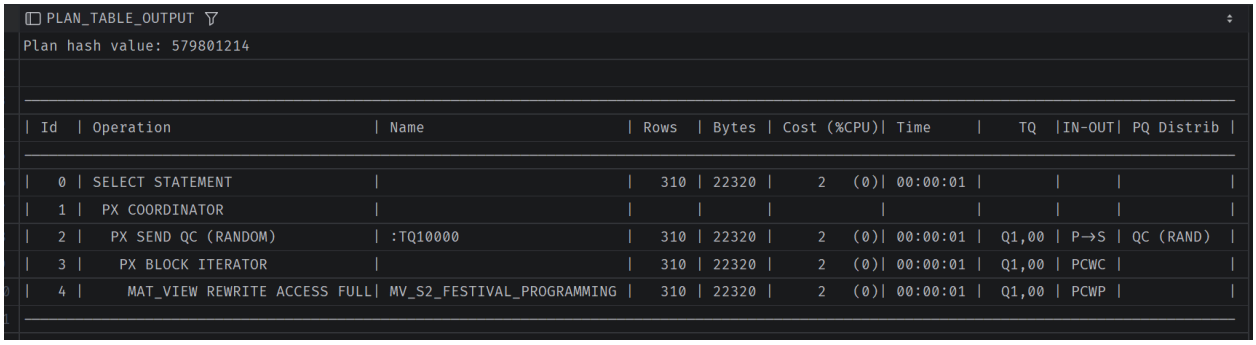


	MVIEW_NAME	REFRESH_METHOD	REFRESH_MODE	REWRITE_ENABLED	COMPILE_STATE	STALENESS
1	MV_S2_FESTIVAL_PROGRAMMING	FAST	COMMIT	Y	VALID	FRESH

FAST | COMMIT | Y | VALID | FRESH = de gewenste configuratie.

Explain plan NA optimalisatie

Dezelfde query, ongewijzigd, wordt nu door query rewrite getransformeerd tot één enkele MAT_VIEW REWRITE ACCESS FULL op MV_S2_FESTIVAL_PROGRAMMING. De vier full table scans en drie hash joins zijn verdwenen.



PLAN_TABLE_OUTPUT

```
Plan hash value: 579801214

-------------------------------------------------------------------------------------------------------------------
| Id  | Operation                     | Name                        | Rows  | Bytes | Cost (%CPU)| Time     |    TQ  |IN-OUT| PQ Distrib |
-------------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT              |                             |   310 | 22320 |     2   (0)| 00:00:01 |        |      |            |
|   1 |  PX COORDINATOR               |                             |       |       |            |          |        |      |            |
|   2 |   PX SEND QC (RANDOM)         | :TQ10000                    |   310 | 22320 |     2   (0)| 00:00:01 |  Q1,00 | P->S | QC (RAND)  |
|   3 |    PX BLOCK ITERATOR          |                             |   310 | 22320 |     2   (0)| 00:00:01 |  Q1,00 | PCWC |            |
|   4 |     MAT_VIEW REWRITE ACCESS FULL| MV_S2_FESTIVAL_PROGRAMMING |   310 | 22320 |     2   (0)| 00:00:01 |  Q1,00 | PCWP |            |
-------------------------------------------------------------------------------------------------------------------
```

Vergelijking Voor/na

```
Optimizer cost      : 2860 ->     2   (~1430x goedkoper)
Wall-clock cold     : 550 ms ->  87 ms (~6x sneller)
Wall-clock hot      : 105 ms ->  26 ms (~4x sneller)
Resultatenset       : 310 rijen (ongewijzigd)
```

Negatieve gevolgen

1. Opslag-overhead

De MV en de vier MV-logs nemen extra disk-ruimte in. Bij creatie is dit minimaal (de MV zelf is 0.063 MB; MV-logs zijn nog niet geallocateerd omdat Oracle deferred segment creation gebruikt). Naarmate DML op de bron-tabellen plaatsvindt, groeien de logs proportioneel met het aantal wijzigingen.

	SEGMENT_NAME	SEGMENT_TYPE	MB
1	MV_S2_FESTIVAL_PROGRAMMING	TABLE	0.063

2. DML-cost (tragere inserts)

Onder REFRESH FAST ON COMMIT betaalt elke transactie die een bron-tabel wijzigt een extra prijs: de MV-log moet bijgewerkt worden én de incrementele refresh moet binnen de commit lopen.

1000 rijen invoegen + COMMIT mét MV actief:

```
[2026-05-19 14:27:12] PROJECT> DECLARE
                                   t_start TIMESTAMP;
                                   t_end   TIMESTAMP;
                               BEGIN
                                   t_start := SYSTIMESTAMP;

                                   INSERT INTO Performance
                                       (Performance_Start_Time, Artist_ID, Performance_End_Time,
                                        Performance_Description, Genre_ID, Stage_ID)
                                   SELECT  SYSDATE + LEVEL/1440,
                                           (SELECT MIN(Artist_ID) FROM Artist),
                                           SYSDATE + LEVEL/1440 + 1/24,
                                           'M7 negative-consequence load test',
                                           (SELECT MIN(Genre_ID)  FROM Genre),
                                           (SELECT MIN(Stage_ID)  FROM Stage)
                                   FROM    dual
                                   CONNECT BY LEVEL <= 1000;

                                   COMMIT;

                                   t_end := SYSTIMESTAMP;
                                   DBMS_OUTPUT.PUT_LINE('Insert + commit duration WITH MV: ' ||
                                                        TO_CHAR(EXTRACT(SECOND FROM (t_end - t_start)), 'FM9990.000') ||
                                                        ' seconds');
                               END;
[2026-05-19 14:27:12] completed in 438 ms
Insert + commit duration WITH MV: 0.407 seconds
```

Dezelfde insert + commit zonder MV (MV tijdelijk gedropt):

```
[2026-05-19 14:28:25] PROJECT> DECLARE
                                   t_start TIMESTAMP;
                                   t_end   TIMESTAMP;
                               BEGIN
                                   t_start := SYSTIMESTAMP;

                                   INSERT INTO Performance
                                       (Performance_Start_Time, Artist_ID, Performance_End_Time,
                                        Performance_Description, Genre_ID, Stage_ID)
                                   SELECT  SYSDATE + LEVEL/1440,
                                           (SELECT MIN(Artist_ID) FROM Artist),
                                           SYSDATE + LEVEL/1440 + 1/24,
                                           'M7 negative-consequence load test',
                                           (SELECT MIN(Genre_ID)  FROM Genre),
                                           (SELECT MIN(Stage_ID)  FROM Stage)
                                   FROM    dual
                                   CONNECT BY LEVEL <= 1000;

                                   COMMIT;

                                   t_end := SYSTIMESTAMP;
                                   DBMS_OUTPUT.PUT_LINE('Insert + commit duration WITHOUT MV: ' ||
                                                        TO_CHAR(EXTRACT(SECOND FROM (t_end - t_start)), 'FM9990.000') ||
                                                        ' seconds');
                               END;
[2026-05-19 14:28:25] completed in 185 ms
Insert + commit duration WITHOUT MV: 0.150 seconds
```

```
Met    MV : 0.438 seconden
Zonder MV : 0.150 seconden
Overhead  : ~3X trager voor write-zware werkbelasting
```

3. Staleness wanneer de auto-refresh uitgeschakeld is

Om aan te tonen wat REFRESH FAST ON COMMIT achter de schermen doet, schakel ik de MV tijdelijk over naar REFRESH FAST ON DEMAND. Daarna volg ik één specifiek festival (Festival_ID = 1) door de hele cyclus.

A: snapshot Voor de probe-insert (Performance_Count = 1924):

	FESTIVAL_ID	PERFORMANCE_COUNT	TOTAL_MINUTES
1	1	1924	234660

B: Na een probe-insert + COMMIT is de MV stale (staleness = NEEDS_COMPILE):

	MVIEW_NAME	STALENESS	LAST_REFRESH_DATE
1	MV_S2_FESTIVAL_PROGRAMMING	NEEDS_COMPILE	2026-05-19 12:30:05

C: Dezelfde SELECT levert nog altijd het oude aggregaat (1924), ondanks de extra rij in de bron-tabel:

	FESTIVAL_ID	PERFORMANCE_COUNT	TOTAL_MINUTES
1	1	1924	234660

D: Na een manuele DBMS_MVIEW.REFRESH consumeert Oracle de MV-log entries en wordt Performance_Count 1925 (oude waarde + 1):

	FESTIVAL_ID	PERFORMANCE_COUNT	TOTAL_MINUTES
1	1	1925	234720

E: De MV staat opnieuw op staleness = FRESH:

	MVIEW_NAME	STALENESS	LAST_REFRESH_DATE
1	MV_S2_FESTIVAL_PROGRAMMING	FRESH	2026-05-19 12:33:39

Onder REFRESH FAST ON COMMIT was deze staleness-vensteruur nooit voorgekomen, Oracle had de log-entries binnen de commit zelf geconsumeerd. Dit is precies waarom ON COMMIT zo waardevol is.

4. Bulk-loads en "stille" staleness

Een vierde, subtieler negatief gevolg ontdekte ik bij het opnieuw vullen van de database. Bulk-load procedures (PKG_S2_festivals.generateRandomPerformances met FORALL INSERT) commiten niet tussentijds; bij de uiteindelijke commit moet ON COMMIT alle 93 200 log-entries in één keer toepassen. Op Oracle Autonomous DB werd dit soms NIET volledig afgehandeld: de rijen zaten wel in Performance, de MV-log had 93 200 entries, maar de MV bleef op het oude aggregaat staan terwijl staleness onterecht FRESH rapporteerde.

Mitigatie: na bulk loads expliciet DBMS_MVIEW.REFRESH(method => 'F') aanroepen als safety net. Voor normale, kleinere transacties werkt ON COMMIT zonder problemen.

Verificatie na herladen van de data

De assignment vereist dat na opnieuw vullen van de database de query nog steeds via de MV de juiste resultaten levert.

Performance is gegroeid van 559 240 naar 652 440 rijen. Na een manuele catch-up refresh komen MV en tabel exact overeen:

	MV_TOTAL	TABLE_TOTAL
1	652440	652440

Het explain plan blijft de MV gebruiken, cost = 2, MAT_VIEW REWRITE ACCESS FULL, ondanks 93 200 nieuwe rijen in de bron:

```
PLAN_TABLE_OUTPUT
Plan hash value: 579801214

-------------------------------------------------------------------------------------------------------------------------------
| Id  | Operation                     | Name                       | Rows  | Bytes | Cost (%CPU)| Time     |    TQ  |IN-OUT| PQ Distrib |
-------------------------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT              |                            |   310 | 22320 |     2   (0)| 00:00:01 |        |      |            |
|   1 |  PX COORDINATOR               |                            |       |       |            |          |        |      |            |
|   2 |   PX SEND QC (RANDOM)         | :TQ10000                   |   310 | 22320 |     2   (0)| 00:00:01 |  Q1,00 | P->S | QC (RAND)  |
|   3 |    PX BLOCK ITERATOR          |                            |   310 | 22320 |     2   (0)| 00:00:01 |  Q1,00 | PCWC |            |
|   4 |     MAT_VIEW REWRITE ACCESS FULL| MV_S2_FESTIVAL_PROGRAMMING |   310 | 22320 |     2   (0)| 00:00:01 |  Q1,00 | PCWP |            |
-------------------------------------------------------------------------------------------------------------------------------
```

Aanvullend bewijs dat de MV opnieuw FRESH is na de refill:

	MVIEW_NAME	REFRESH_METHOD	REFRESH_MODE	STALENESS	LAST_REFRESH_DATE
1	MV_S2_FESTIVAL_PROGRAMMING	FAST	COMMIT	FRESH	2026-05-19 13:05:11

Conclusie

De materialized view met REFRESH FAST ON COMMIT en ENABLE QUERY REWRITE levert voor het festival-programming-dashboard een dramatische verbetering: de optimizer-kost daalt van 2860 naar 2 (~1430× goedkoper), de cold-cache uitvoeringstijd van 550 ms naar 87 ms (~6× sneller), en query rewrite zorgt dat dit volledig transparant gebeurt, geen applicatiecode hoeft te veranderen.

De prijs voor deze leessnelheid is een trager schrijfpad: inserts op Performance worden ~3× trager omdat elke commit ook een incrementele MV-refresh moet uitvoeren. Voor read-heavy workloads (zoals een dashboard) is dat een acceptabele trade-off.

Het refresh-mechanisme bestaat uit twee gekoppelde lagen: MV-logs op de vier bron-tabellen (met ROWID, SEQUENCE en INCLUDING NEW VALUES) leggen de wijzigingen vast, en REFRESH FAST ON COMMIT past die wijzigingen incrementeel toe binnen de transactie zelf. Onder normale DML werkt dit automatisch en houdt het de MV continu fris. Voor uitzonderlijke bulk-load scenarios met grote log-backlogs is een manuele DBMS_MVIEW.REFRESH als safety net nodig, geïmplementeerd in S2_OPT_refill.sql.